eedi-mining-misconceptions-in-mathematics

8th Place Solution

Rank	8
Team	Yannan Chen
Author	Yannan Chen
Topic ID	551412
Version	Original

Source: https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/551412
Retrieved with Kaggle CLI on 2026-10-21.

After reading the top solutions, I'm truly impressed by other teams' dedicated efforts, great attention to training & inference details, and highly sophiscated methods of prompt engineering. As a novice, I feel fortunate to have achieved my final ranking and am humbled to share my solution. I think my solution, yet simple, but is still effecitve and comparatively computing efficient. I used only 14B and 32B models, with a total running time of about 6 hours.

I'd like to extend my gratitude to everyone who shared their code and insights, both during and after the competition.

Two key references

- @sayoulala : https://www.kaggle.com/competitions/eedi-mining-misconceptions-in-mathematics/discussion/543519
  - Enlightened me on LoRA finetuning, hard negative mining, and recall+rerank training scheme.
- @cdeotte : https://www.kaggle.com/competitions/lmsys-chatbot-arena/discussion/521294
  - Inspiring ideas on model quantization, wise-FT LoRA adaptation, and VLLM inference.

Inference pipeline

retrievers (4 Qwen2.5-14b models ensembled by weighted sum) → 40 misconceptions → reranker (a single Qwen2.5-32b model)→ 25 misconceptions

Training details

In general, I find iterative hard negative mining very helpful.

- Use few-shots prompt with chatgpt to generate extra training data:
  - Added 983 synthetic data samples to fill missing misconceptions.
  - training data is split in 5 CV-folds by `"QuestionId"`. For each fold, regenerated extra samples to cover unique misconceptions in the valid fold for training.
- Retriever (Qwen2.5-14b): Finetuned with bnb bit4 + LoRA for FEATURE_EXTRACTION task.
  - trained on 2 A100s.
  - lora parameters: `r=32, alpha=64, target_modules=['q_proj','k_proj','v_proj','o_proj','gate_proj','up_proj','down_proj']`
  - training paramters: `train_group_size=4, effective_batch_size=64, epoch=7, lr_scheduler_type="cosine"`;
  - trained in 5 CV-folds.
    - For first 4~5 epochs, model is trained without generated data, with a focus on improving map25.
    - For the final 2~3 epochs, generated data is added into training, which helps to further improve map25 and recall25.

    - I interfere manually when the valid scores (map25 & recall25) stop improving by following means:
        - reducing learning_rate gradually from 5e-4 to 1e-5
        - reducing negative data pool size from 80 to 4, for more focus on distinguishing hardest misconceptions
- Reranker (Qwen2.5-32b): Finetuned with bnb bit4 + LoRA for CASUAL_ML task, outputting a “Yes” logit for each query–misconception pair.
    - trained on 2 A100s.
    - lora parameters: `r=32, alpha=64, target_modules=['q_proj','k_proj','v_proj','o_proj','gate_proj','up_proj','down_proj','down_proj','lm_head']`
    - training paramters: `train_group_size=4, effective_batch_size=64, epoch=4, learning_rate=5e-4, lr_scheduler_type="WarmupDecayLR"`;
    - full dataset used for training reranker without validation. Negative data is generated by 4 retrievers (emsembled through weighted sum) with negative_count = 40

Quantization & Inference details

- Retrieving:
    - 4 retrievers (chosen from 5-fold CV based on LB scores) retrieve 4 times, then ensembled via weighted sum.
    - infer in bnb bit4+LoRA (no need to merge), so that each single retriever can fit in one T4 gpu, 4 retrievers only take 2 T4s running twice.
- Reranking:
    - model is first merged with lora adaptor, with wise-FT hyper parameter alpha=0.8
    - model is then quantized to AWQ (calibrated using 128 samples randomly selected from training data)
    - VLLM is used for final inference. Each query is paired with 40 misconceptions retrieved, generating 40 queries for reranker to score. "enable_prefix_caching" is turned on to accerlerate inference.

Key improvements (Public LB)

- Add Synthetic data: single retriever public LB score from 0.3~ → 0.49
- Ensemble retrievers: 0.49 → 0.51
- Add reranker: 0.51 → 0.57
- Apply Wise-FT on reranker's LoRA merging: 0.57 → 0.59

Personal reflections

The entire pipeline — from LoRA finetuning to quantization — was very new to me, and I nearly gave up more than once. I started small with Qwen2.5-0.5B to establish a workable process, but the AWQ-quantized model kept producing random letters during inference. After much frustration, I discovered that autoawq didn't export the im-head-embed layer parameters for Qwen2.5-0.5B, forcing me to add them manually. Without @cdeotte's paper proving it could be done, I might have quit altogether. Thanks to that reassurance, I stuck with it and eventually got the pipeline running smoothly.

Code

- Inference code: https://www.kaggle.com/code/yannan90/eedi-recall-rerank-lb-0-601-pb-0-584
- Training code: https://github.com/yannan90/kaggle-2024-Eedi-8th-place-training-code

Supplemental Q&A

> **Nikhil Tumbde · 2024-12-13T12:58:50.693000**
>
> Thanks for the solution. The enable_prefix_caching parameter and wise-FT were new to me. I have few questions for you as this is a very new space for me, it would be really helpful if you could help me understand (maybe please include these pointers in your next writeup).
>
> 1. can you share the prompts (or some idea around it) used for synthetic data generation? Also, did you generate the data for misconceptions not seen in the training data
> "Added 983 synthetic samples to fill missing misconceptions."
> 2. can you also share how to do this manually as I also struggled with the same ?
>
> > After much frustration, I discovered that autoawq didn't export the im-head-embed layer parameters for Qwen2.5-0.5B, forcing me to add them manually.>

> **Yannan Chen · 2024-12-13T13:24:29.493000**
>
> 1. yes, I generated samples for misconceptions not seen in the training data. When doing OOF training, the fold used for validation also contains some unique misconceptions, I generated for that too. The prompt I used was built in my native language but the key components are:
>
> - explain the task and format of each new data sample you want to AI to generate (ConstructName, SubjectName, QuestionText, CorrectAnswerText…)
> - give 3~4 examples randomly selected from existing train data as example.
> - give the new misconceptions you want AI to generate based upon. (could be one, or a list)
> - remind AI to not miss any misconceptions, return back proper format, and any other limitations you want to add.

> **Yannan Chen · 2024-12-13T13:32:23.177000**
>
> 2.This is a very weird issue. Here is how to manually adjust the params file:

> the safetensors file can be loaded as just simple dictionary format using lib `from safetensors.torch import safe_open, load_file, save_file`, with its keys being the name of each layer, and the values being the tensors.
>
> So after the file is loaded, you just need to compare with the un-quantized model's params (loaded in as dict too), see if there is any mismatch in keys. If so, modify the dict by code like `quant_params[k] = unquant_params.get(k)`. Eventually save the modified params back.